

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: import data using transform maps(spreadsheet)

Team Members:

- [Student Name 1] - Frontend Developer
- [Student Name 2] - Backend Developer
- [Student Name 3] - Database Administrator

Team ID: SWTID-2026-7220

Date: 30 September 2026

2. Project Overview

Purpose: The purpose of this project is to provide a seamless and dynamic interface for users to upload spreadsheet files (CSV, Excel), map the columns of the uploaded file to target database schemas using transform maps, and securely import the data. It eliminates manual data entry and prevents data format mismatches.

Features:

- Drag-and-drop file upload for spreadsheets (.csv, .xlsx).
- Interactive UI for mapping spreadsheet columns to database fields.
- Data validation and error reporting before final import.
- Save transform maps as templates for future uploads.
- Dashboard for viewing import history and logs.

3. Architecture

Frontend: Developed using React.js. It utilizes functional components, React Hooks for state management, and Axios for handling API calls. The UI is built with a focus on user experience during the column mapping phase.

Backend: Built with Node.js and Express.js. It handles REST API requests, processes uploaded files using libraries like `multer` and `xlsx`, and manages the business logic for data transformation based on the maps provided by the client.

Database: MongoDB is used as the primary database. The schema includes collections for Users, TransformMaps (storing mapping rules), and ImportedData (the final destination for the spreadsheet rows).

4. Setup Instructions

Prerequisites:

- Node.js (v16.x or higher)
- MongoDB (Local instance or MongoDB Atlas URI)
- Git

Installation:

```
# 1. Clone the repository git clone https://github.com/[your-repo]/import-transform-maps.git cd import-transform-maps # 2. Install Server Dependencies cd server npm install # 3. Configure Server Environment Variables # Create a .env file in the server directory PORT=5000 MONGO_URI=your_mongodb_connection_string JWT_SECRET=your_secret_key # 4. Install Client Dependencies cd ../client npm install
```

5. Folder Structure

Client (React):

```
client/ |— public/ |— src/ | |— components/ # Reusable UI components (Uploader, Mapper) | |— pages/ # Page layouts (Dashboard, UploadPage) | |— services/ # API integration (Axios calls) | |— App.js | |— index.js
```

Server (Node.js):

```
server/ |— controllers/ # Business logic (uploadController, mapController) |— models/ # Mongoose schemas (User, TransformMap) |— routes/ # API endpoints definition |— middleware/ # Auth and file upload middlewares |— server.js # Entry point
```

6. Running the Application

Provide commands to start the frontend and backend servers locally.

Backend: Open a terminal, navigate to the `server` directory, and run:

```
npm start
```

Frontend: Open a new terminal, navigate to the `client` directory, and run:

7. API Documentation

- **POST /api/upload** - Uploads the spreadsheet file.
Parameters: FormData containing the `file`.
Response: Returns parsed headers/columns and sample data for mapping.
- **POST /api/maps/save** - Saves a user-defined column mapping template.
Parameters: { mapName, mappings: [{ sourceCol, targetCol }] }.
Response: Success message and saved map ID.
- **POST /api/import** - Executes the data import based on the transform map.
Parameters: { fileId, mapId }.
Response: Number of rows successfully imported and error logs (if any).

8. Authentication

Authentication is handled using JSON Web Tokens (JWT). When a user logs in, the backend issues a signed JWT which the client stores in LocalStorage. All protected API routes (like uploading files and saving maps) require the token to be sent in the `Authorization: Bearer <token>` header.

9. User Interface

[Placeholder: Insert Screenshots of Dashboard]

[Placeholder: Insert Screenshots of Drag-and-Drop Uploader]

[Placeholder: Insert Screenshots of Column Mapping Interface]

10. Testing

The application employs a combination of testing strategies. **Backend:** Unit testing for API routes and controllers is conducted using Jest and Supertest. **Frontend:** React components, specifically the mapping logic, are tested using React Testing Library to ensure UI reliability.

11. Screenshots or Demo

Project Demo Video / Live URL: [\[Insert Link Here\]](#)

12. Known Issues

- Processing extremely large files (over 50MB) synchronously may cause the request to time out. Future iterations will require a background worker (e.g., BullMQ).
- Date formats from Excel files sometimes require manual adjustment depending on the user's system locale.

13. Future Enhancements
